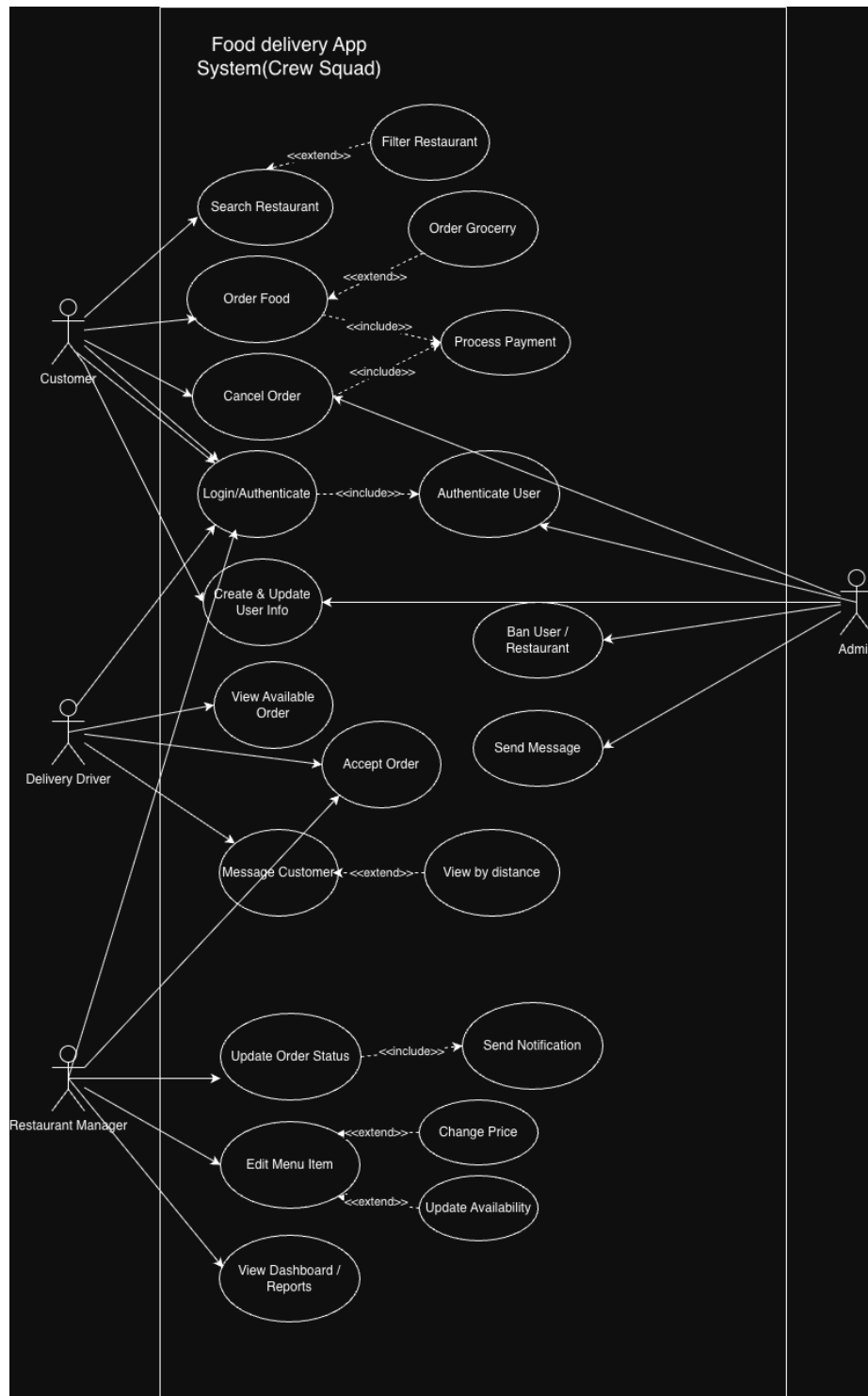


CrewSquad COSC 310 Project M2

Use Case Diagram:



Chosen Architecture Pattern: Layered Architecture (N-Tier)

To support the complex interactions between actors such as (Customers, Restaurant Managers, Delivery Drivers) and the diverse sets of (Ordering, Payments, Notifications), we have selected a Layered Architecture for our Architecture Pattern. This design will help by organizing the system into four distinct horizontal layers, where each layer has a specific responsibility and interacts strictly with the layer beneath it. This separation of concerns ensures the system is modular, testable, and secure. The design will be split up into the following layers:

1. Presentation (API): Handles HTTP requests and Authentication.
2. Business Logic (Services): Encapsulates core domain rules.
3. Data Access (Repositories): Abstracts database interactions.
4. Persistence (Database): Stores data.

To ensure the Layered Architecture meets our requirements, we made the following implementation decisions in regards to our past functional and non-functional requirements:

1. Isolating Logic in the Service Layer (Supporting Feature 4)

- We will strictly enforce that all order workflows (Pending -> Preparing -> Delivered) happen only in the Service Layer and not in the API routes. This guarantees that Feature 4 (Order Consistency) is upheld by preventing prohibited status changes and ensures that the business rules are always applied correctly.

2. Enforcing Data Integrity with Repositories (Supporting Feature 2)

- We chose to use a Repository Pattern to handle all database connections. This ensures that Feature 2 (Menu Integrity) will be satisfied because the repository acts as a gatekeeper, checking that a menu item is valid and linked to a restaurant before it comes in contact with the database.

3. Centralizing Security in Middleware (Supporting Feature 1)

- We implemented a shared Authentication Middleware that sits between the API and Service layers. This will meet Feature 1 (Role-Based Access Control) by making every request verify JWT tokens, ensuring that a Customer cannot access Restaurant Manager functions.

4. Using Asynchronous I/O (Supporting NFR-1 on M1)

- We decided to build the Presentation Layer using FastAPI's asynchronous capabilities. This ensures the system meets the Non-Functional Requirement of responding to requests within 300ms. It allows the server to handle high traffic without freezing while waiting for database responses.

5. Containerizing the Stack (Supporting NFR-6 on M1)

- We defined the entire layered stack in docker-compose. This meets the deployment requirement (NFR-6) by ensuring the API, Database, and all dependencies launch with a single command on any operating system.

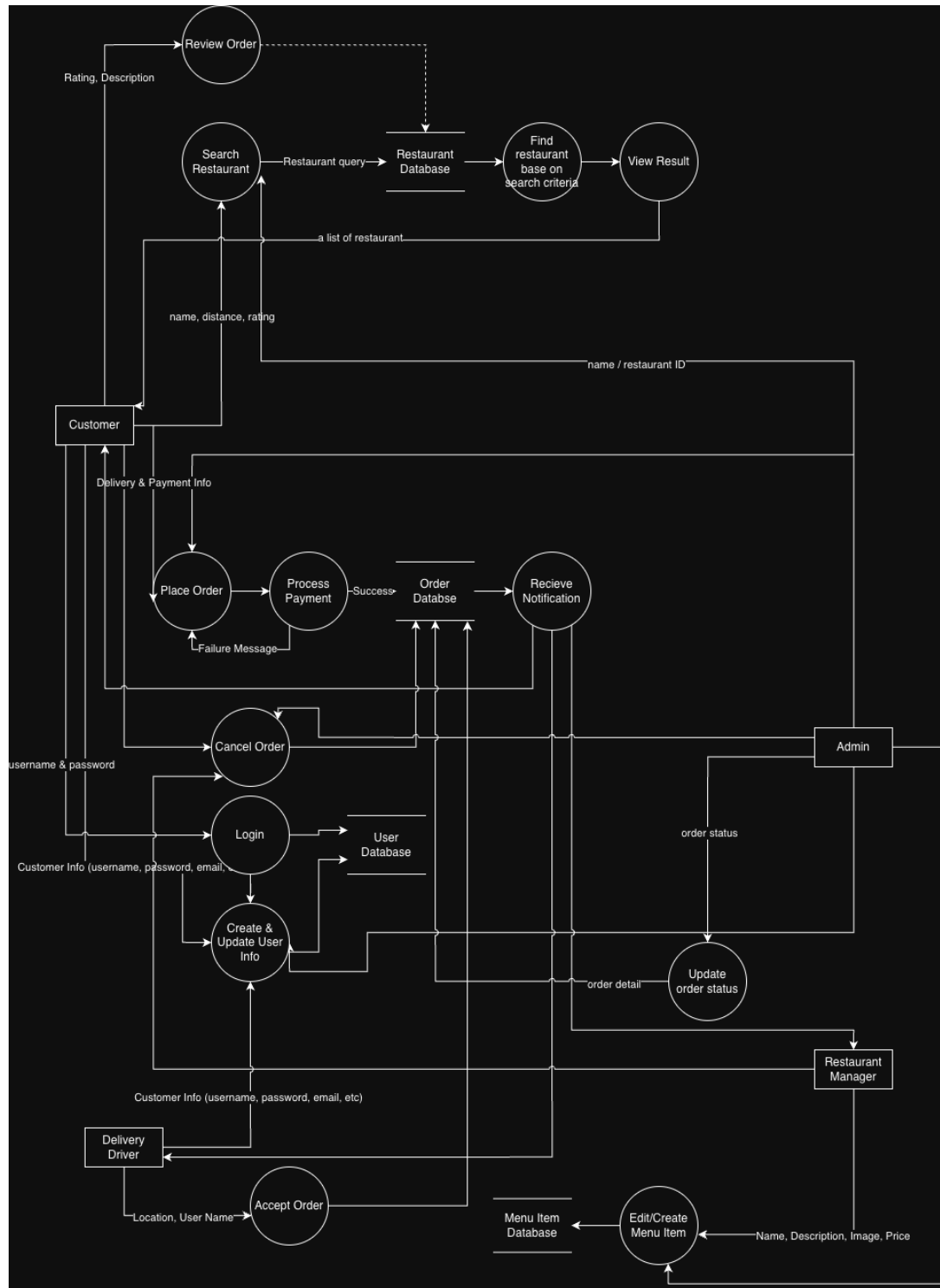
Component Identification:

Component	Responsibilities (What it does)	Key Classes / Files	Interactions
Authentication Service	Manages user identity, including registration, login, password hashing etc. It issues and validates JWT tokens to control access.	AuthRouter, AuthService, User, Token	Used to verify if a user is authorized before processing requests.
Restaurant Service	Manages restaurant profiles, menus, and item details. It enforces rules like "Menu items must belong to a valid restaurant" and prevents duplicate entries, or other similar issues.	MenuRouter, RestaurantService, Restaurant, MenuItem	Accessed by the Order Service to validate that items exist and that their prices are correct before an order is placed.

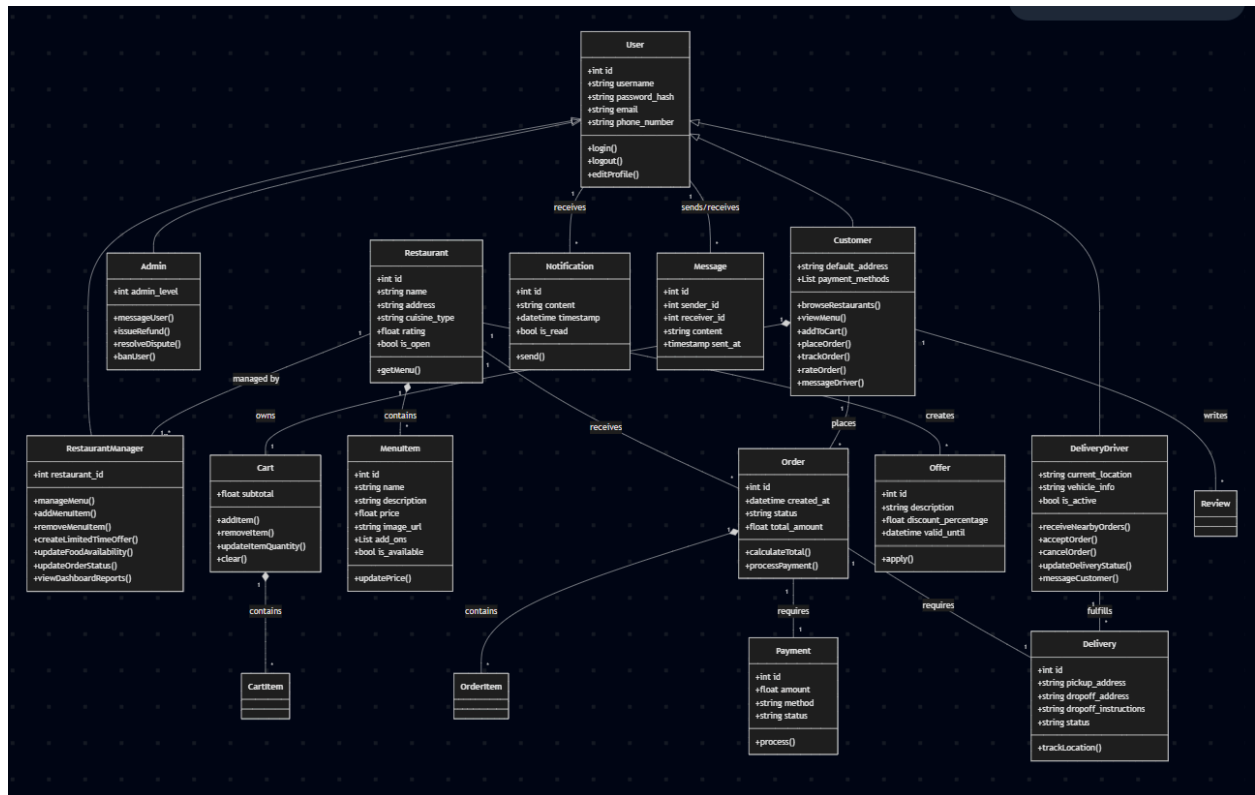
Search Service	Handles high-performance searching and filtering (by Cuisine, Name, Price).	SearchRouter, SearchService, RestaurantRepository	Connects directly to the Restaurant Repository for read-only access. It does not modify data, ensuring speed.
Order Service	Acts as the "brain" of the transaction system. It manages the Order Lifecycle (Pending, Preparing, Delivered) and calculates totals (including tax/fees).	OrderRouter, OrderService, OrderRepository	Calls the Payment Service to initiate a transaction and triggers the Notification Service whenever an order status changes.
Payment Service	Simulates a third-party payment gateway. It randomly "Approves" or "Declines" transactions to mimic real-world scenarios and records the outcome.	PaymentRouter, PaymentSimulator, Transaction	Updates the Order Service with the payment result (e.g., marks order as PAID or FAILED).

Notification Service	Generates system events and alerts for users (e.g., "Order Accepted"). It maintains a log of all notifications for the user's history.	NotificationService, EventLog	Listens for state changes from the Order Service and pushes updates to the relevant User or Restaurant Owner.
Delivery Service	Manages delivery logistics. It broadcasts new orders to available drivers, validates user delivery addresses, and handles driver assignments. It ensures that changing a user's location does not affect active orders.	DeliveryRouter, DeliveryService, Driver, LocationValidator	Interacts with the Order Service to get order details and destination. Updates the order status when a driver picks up or completes a delivery.
Pricing Service	Handles all financial calculations, mainly for discounts.	PricingService, CostCalculator, TaxStrategy, FeeManager	Called by the Order Service during the checkout process to generate a quote before the user confirms the order.

Data Flow Diagram

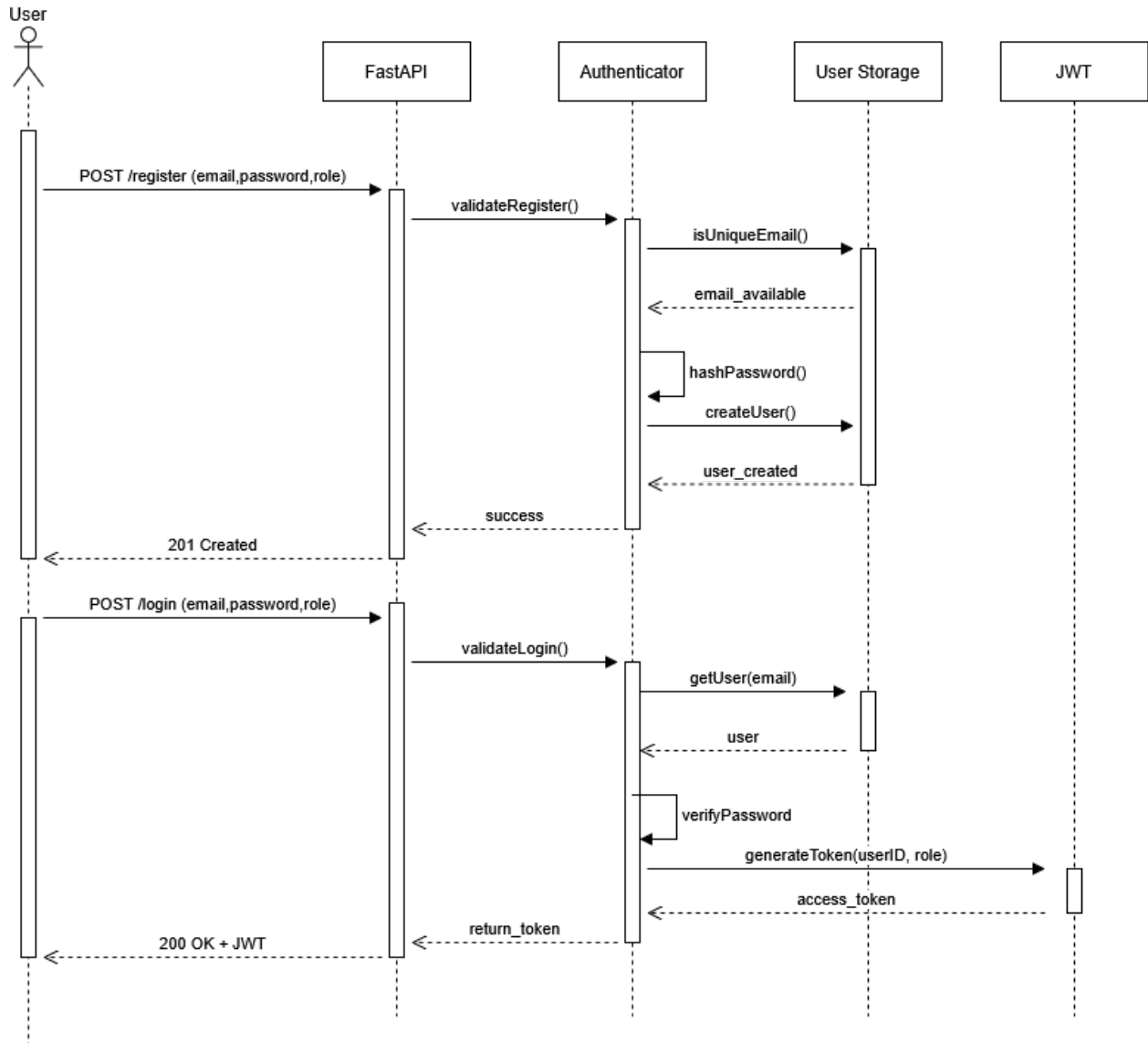


Class Diagram

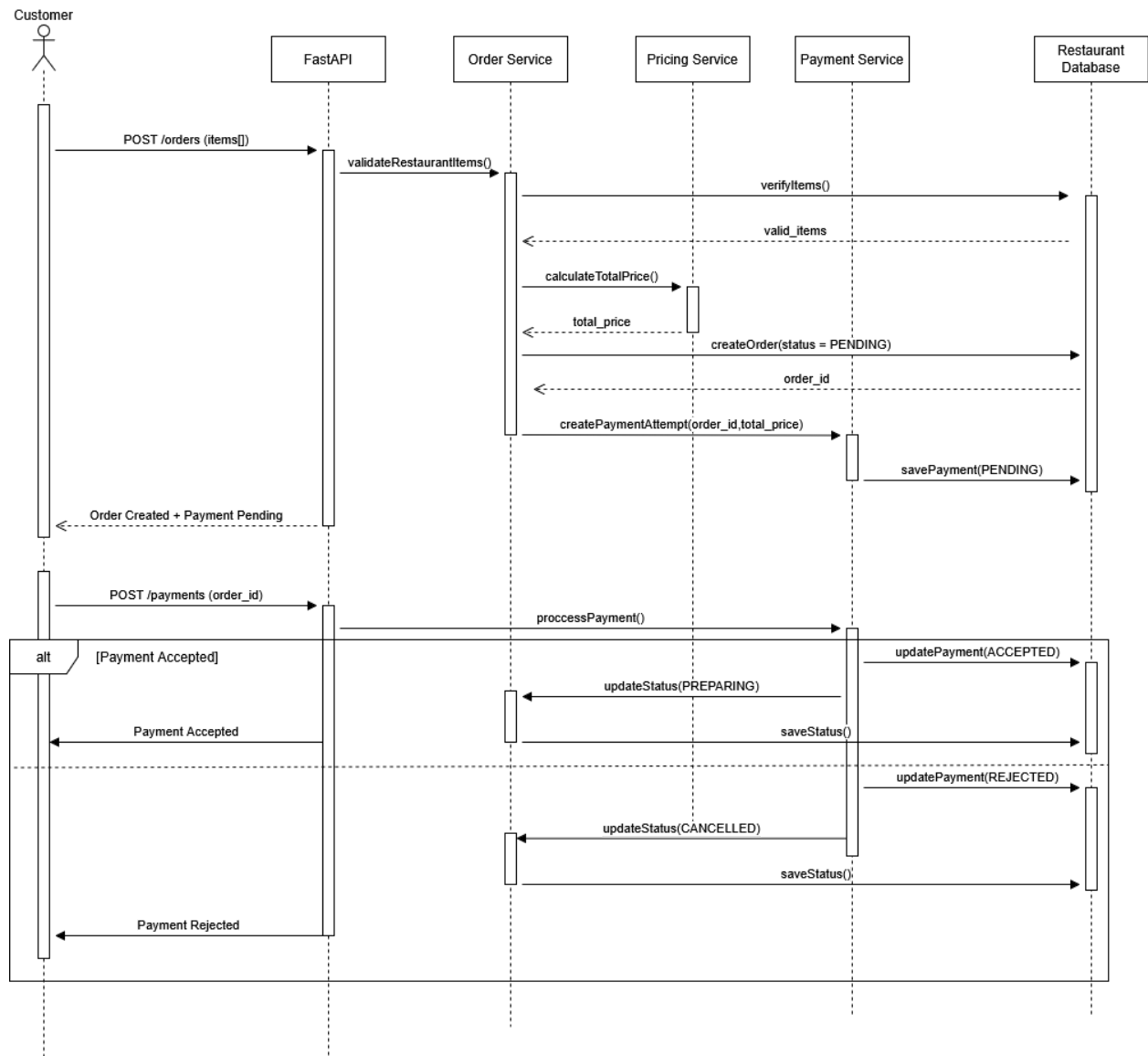


Sequence Diagrams

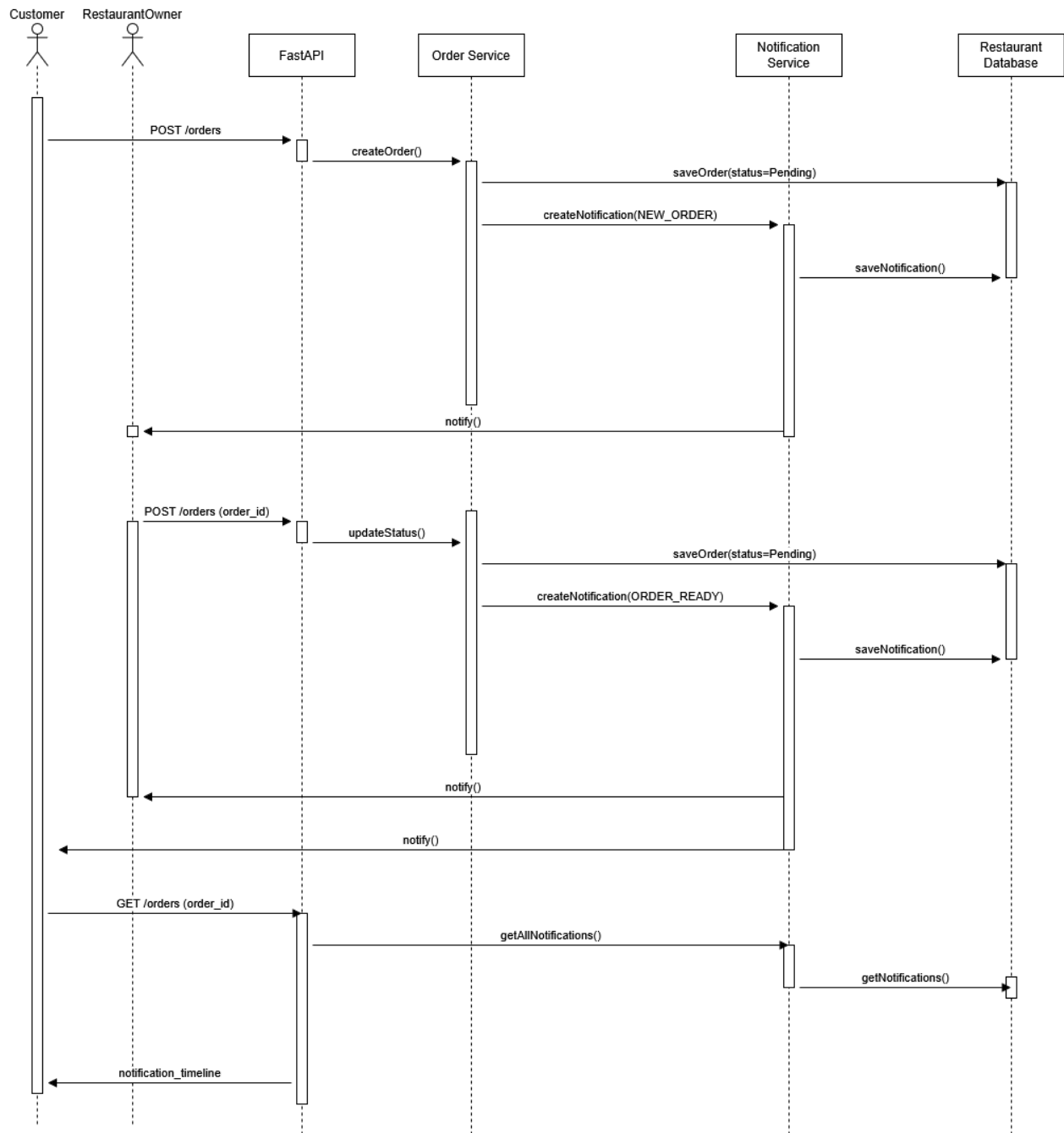
Register and Login Sequence Diagram



Order and Payment Sequence Diagram



Status and Notification Sequence Diagram



Traceability Matrix:

CF IDs	Core Features	FR IDs	Functional Requirements	US IDs	User Stories	System Components
Feat 1	The system will allow users and restaurant owners/managers to create accounts and log in. It will handle user authentication (login), authorization (what each user is allowed to do), and basic user identity management. Different roles such as regular users and restaurant owners/managers will be supported.	Feat 1-FR 1	The system shall provide an endpoint for users to register with a unique email, password, and role (Customer or Restaurant Owner).	Feat 1-US 1	As a new customer, I want to create an account with my email and password so that I can save my delivery address and order history.	Authentication Service
		Feat 1-FR 2	The system shall authenticate users using JWT (JSON Web Tokens) to maintain secure sessions.	Feat 1-US 2	As a returning user, I want to log in securely so that I can access my personal dashboard and shopping cart.	
		Feat 1-FR 3	The system shall enforce Role-Based Access Control (RBAC), ensuring only Restaurant Owners can modify menu data.	Feat 1-US 3	As a restaurant owner, I want to register my business so that I can manage my restaurant's digital storefront and menu.	
		Feat 1-FR 4	The system shall securely hash passwords	Feat 1-US 4	As a user, I want my password to be encrypted so that my personal	

			before storing them in the database.		account remains secure even if the database is compromised.	
Feat 2	The system will store information about restaurants and their menus. It will ensure that data is valid, properly connected (for example, menu items must belong to a restaurant), and that basic constraints are enforced, such as preventing invalid or missing values.	Feat 2-FR 1	The system shall allow Restaurant Owners to add, update, or delete menu items associated with their specific restaurant ID.	Feat 2-U S1	As a restaurant owner, I want to add new dishes to my menu with prices and descriptions so that customers know what I am currently serving.	Restaurant Service
		Feat 2-FR 2	The system shall validate that every menu item contains a name, price, and category (ex. Appetizer, Main, Drink).	Feat 2-U S2	As a customer, I want to view a restaurant's full menu organized by categories so that I can easily find the type of food I'm looking for.	
		Feat 2-FR 3	The system shall validate item prices and restaurant cuisine types to ensure data consistency.	Feat 2-U S3	As a restaurant owner, I want to ensure my menu prices are accurate so that I don't lose money on orders.	
		Feat 2-FR 4	The system shall maintain referential integrity, ensuring no menu items exist without a	Feat 2-U S4	As a customer, I want to see that every dish in the app belongs to an active, valid restaurant, so that I don't order	

			parent restaurant.		items that cannot be delivered.	
Feat 3	The system will allow users to browse restaurant menus and search for items or restaurants. Backend logic will handle filtering, searching, and returning paginated results	Feat 3-FR 1	The system shall provide an API endpoint that allows users to filter restaurants specifically by name, cuisine type, or minimum rating.	Feat 3-US1	As a hungry customer, I want to search for restaurants by name, cuisine type, or rating, so that I can quickly find a place that matches my specific wants.	Search Service
		Feat 3-FR 2	The system shall prevent performance issues by limiting the number of results returned in a single request, for example: 10 at a time. Using limit and offset parameters.	Feat 3-US2	As a user with a bad internet connection, I want the system to return restaurant menus in small amounts, so that the application loads quickly and doesn't overwhelm me with data.	
		Feat 3-FR 3	The system shall provide database queries that must support sorting results efficiently by Price as ascending or Rating as descending before	Feat 3-US3	As a budget-conscious student, I want to sort search results by Price (Low to High) or Rating (High to Low), so that I can prioritize restaurants that fit my budget or quality standards.	

			sending them to the user.			
		Feat 3-FR 4	The system shall return a helpful message if the user searches for a type of food and restaurant and there are no results, instead of an error message.	Feat 3-U S4	As a user searching for a specific food, I want to receive a useful message or suggestion if my search yields no results, so that I am not left looking at a blank screen and can find an alternative option.	
Feat 4	The system will allow users to create and manage food orders. It will ensure that orders are consistent, correctly stored, and follow business logic for the domain (for example, an order cannot be modified after it is completed).	Feat 4-FR 1	The system shall validate that every item in a new order belongs to the same restaurant, mixed-restaurant orders will be rejected.	Feat 4-U S1	As a customer, I want to create a new order with multiple items from a single restaurant, so that I can get all my food delivered in one go.	Order Service
		Feat 4-FR 2	The system shall make sure the backend will enforce a strict system where orders can only move forward (Pending, Preparing, Delivered) and cannot skip steps.	Feat 4-U S2	As a restaurant manager, I want to prevent any modifications to an order once it is marked as Completed or Cancelled, so that our financial records and delivery metrics are accurate and straightforward.	

		Feat 4-FR 3	The system shall make sure that once an order reaches a final status like Delivered or Cancelled, the system must lock the record so no further changes can be made.	Feat 4-U S3	As a restaurant manager, I want to view a queue of incoming Pending orders and explicitly accept or reject them, so that I can manage my kitchen's capacity effectively.	
		Feat 4-FR 4	The system shall confirm that the API must ensure that only Restaurant Managers can Accept or Reject orders, while Customers can only Cancel their own orders before preparation starts.	Feat 4-U S4	As a customer, I want to be able to cancel my order if the restaurant hasn't started preparing it yet, so that I can correct a mistake without causing food waste or financial loss.	
Feat 5	The system will manage delivery-related information. It will support assigning deliveries and tracking basic delivery status as part of the backend logic	Feat 5-FR 1	The system shall broadcast the order to drivers that are close to the restaurant.	Feat 5-U S1	As a food delivery driver, I want to be able to filter orders by distance, so I can fulfil my orders more efficiently.	Delivery Service

		Feat 5-FR 2	The system shall validate the location the user enters when setting/updating location.	Feat 5-US 2	As a customer, I want to be able to change my delivery location, so I can order food in other cities.	
		Feat 5-FR 3	The system shall share the location & delivery instructions to the driver.	Feat 5-US 3	As a customer, I want to be able to leave a note on how to drop off the food to drivers (should the driver leave the food somewhere or hand it to the customer, building access code, etc), so the driver won't be confused on arrival.	
		Feat 5-FR 4	The system shall allow users to change their location, but this should not affect ongoing orders.	Feat 5-US 4	As a customer, I want to be able to order from the closest available brand restaurant (like KFC, Subway, etc) by default, so I can get my food fresh.	
Feat 6	The system will calculate the total cost of an order, including item prices, delivery fees,	Feat 6-FR 1	The system shall compute the taxes based on region, delivery fee	Feat 6-US 1	As a customer, I want to be able to view each fee before ordering, so I can know exactly how	Pricing Service

	and taxes. These calculations will follow predefined business rules implemented in the backend.		based on distance and road/weather condition, and apply other additional fees if applicable (like beverage recycle fee).		much I will have to spend on each fee.	
		Feat 6-FR 2	The system shall show all the fees (individual item price, delivery fee, taxes, service fee, beverage fee) to users on checkout.	Feat 6-U S2	As a customer, I want to be able to view each fee after placing the order, so I can keep a record of my purchase.	
		Feat 6-FR 3	The system shall save the order history and allow the user to view it later.	Feat 6-U S3	As a restaurant owner, I want to make the adjusted item price update on the user-end immediately, so when I make limited time offers, users can see it without needing to refresh the page.	
		Feat 6-FR 4	The system shall allow restaurant owners to adjust item prices, but this	Feat 6-U S4	As a customer, I want the system to adjust the delivery fee / service fee based on the	

			should not affect orders that have already been placed.		supply and demand (like raising the delivery fee in bad weather), so orders won't be left unhandled.	
Feat 7	The system will simulate payment processing. No real payment gateway will be used, but the system will follow the correct workflow for accepting or rejecting a payment and updating the order status.	Feat 7-FR 1	The system shall create and store a payment attempt whenever a user submits an order, this includes the order's ID, amount to be spent, with a status of the order. The initial status will be PENDING.	Feat 7-US1	As a restaurant owner, I want to choose to accept or reject an order, so that I can ensure my workers aren't overwhelmed.	Payment Service (Handles the simulated payment)
		Feat 7-FR 2	The system shall give the user the decision to accept or reject a payment attempt with a box for reason.	Feat 7-US2	As a restaurant owner, I want to have the ability to give a reason for rejecting an order, so that I can maintain that customer's loyalty.	
		Feat 7-FR 3	The system shall simulate the payment outcome, via whether the payment was accepted or	Feat 7-US3	As a user, I want my payment to be simulated when I place an order so that I can experience a	

			rejected, and the payment result will either be timestamped with an ACCEPTED status or REJECTED status with a reason given.		realistic checkout process.	
		Feat 7-FR 4	The system shall update the order status based on the given payment outcome and display the result to the user.	Feat 7-U S4	As a user, I want to see that my order and payment have been accepted, so that I know if I have to change restaurants.	
Feat 8	The system will generate notifications or events when important actions occur, such as order creation or status changes.	Feat 8-FR 1	The system shall create and store a notification when an order is made, this includes the notification type, order ID, restaurant ID, and a timestamp.	Feat 8-U S1	As a restaurant owner, I want to be notified when a new order is placed, so that I can respond quickly.	Notification Service (Handles alerts)
		Feat 8-FR 2	The system shall create and store a notification when the order status changes and ensure that it reaches the correct recipients i.e. the user and restaurant owner.	Feat 8-U S2	As a restaurant owner, I want to see a list of recent orders, so that I can track previous orders and ensure how many must be fulfilled.	

		Feat 8-FR 3	The system shall contain an interface that holds a list of notifications for the current order to give the recipient a timeline of events.	Feat 8-U S3	As a user, I want to receive updates when my order status changes, so that I can see what's happening with my food.	
		Feat 8-FR 4	The system shall notify the restaurant owner of a new order as soon as possible so they can respond quickly to the order.	Feat 8-U S4	As a user, I want to receive a notification when my payment goes through, so that I know the order is getting prepared.	

Contribution:

Branden Skoupas: I worked on the architecture overview and the class diagram. 25% contribution.

Keir Parnitzke: I worked on the use case diagram as well as the component identification. 25% contribution.

Zijian Chen: I worked on the dataflow diagram as well as the use case diagram. 25% contribution.

Paul Daoud: I completed the 3 sequence diagrams and worked on the traceability matrix. 25% contribution.

Backlog URL: <https://github.com/users/KeirPar/projects/1>

Repo URL: https://github.com/KeirPar/CrewSquad_310